

Лабораторная работа № 5

Эмуляция и измерение потерь пакетов в глобальных сетях

Шияпова Дарина Илдаровна

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
5	Выводы	17
	Список литературы	18

Список иллюстраций

4.1	Информацию о сетевых интерфейсах и IP-адресах хостов	8
4.2	Проверка соединения между хостами	9
4.3	Добавление потери пакетов	10
4.4	Добавление потери пакетов	11
4.5	Добавление значения корреляции для потери пакетов	12
4.6	Добавление повреждения пакетов	13
4.7	Добавление переупорядочивания пакетов	14
4.8	Добавление дублирования пакетов	14
4.9	Создание скрипта для эксперимента lab_netem_ii.py	15
4.10	Редактирование скрипта	15
4.11	Makefile	16
4.12	Проведение эксперимента	16

1 Цель работы

Основной целью работы является получение навыков проведения интерактивных экспериментов в среде Mininet по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных. Эти параметры влияют на производительность протоколов и сетей.

2 Задание

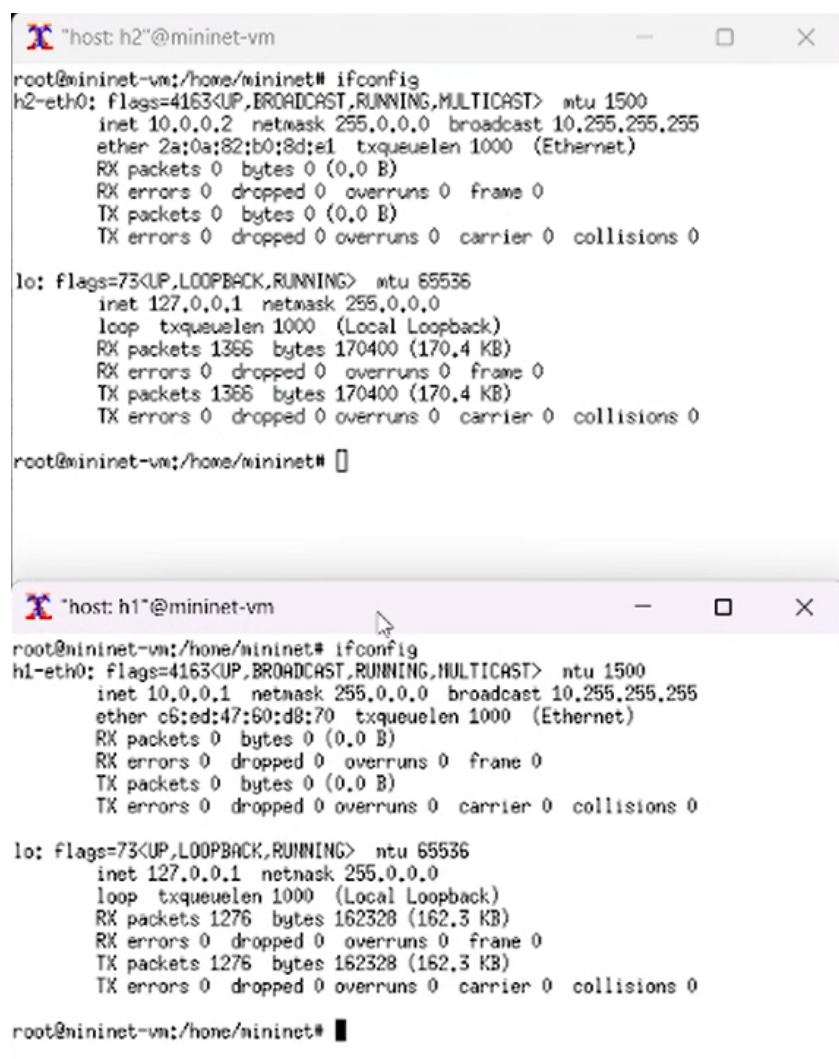
1. Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных.
3. Реализуйте воспроизводимый эксперимент по добавлению правила отбрасывания пакетов в эмулируемой глобальной сети. На экран выведите сводную информацию о потерянных пакетах.
4. Самостоятельно реализуйте воспроизводимые эксперименты по исследованию параметров сети, связанных с потерей, изменением порядка и повреждением пакетов при передаче данных. На экран выведите сводную информацию о потерянных пакетах.

3 Теоретическое введение

Mininet – это эмулятор компьютерной сети. Под компьютерной сетью подразумеваются простые компьютеры – хосты, коммутаторы, а так же OpenFlow-контроллеры. С помощью простейшего синтаксиса в примитивном интерпретаторе команд можно разворачивать сети из произвольного количества хостов, коммутаторов в различных топологиях и все это в рамках одной виртуальной машины(ВМ). На всех хостах можно изменять сетевую конфигурацию, пользоваться стандартными утилитами(`ifconfig`, `ping`) и даже получать доступ к терминалу. На коммутаторы можно добавлять различные правила и маршрутизировать трафик.

4 Выполнение лабораторной работы

Зададим простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8. На хостах h1 и h2 введем команду `ifconfig`, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам. В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы h1-eth0 и h2-eth0 (рис. 4.1).



The image shows two terminal windows from a Mininet VM. The top window is titled "host: h2" and shows the output of the 'ifconfig' command for host h2. It displays the configuration for the 'h2-eth0' interface (IP: 10.0.0.2) and the 'lo' loopback interface (IP: 127.0.0.1). The bottom window is titled "host: h1" and shows the output of the 'ifconfig' command for host h1. It displays the configuration for the 'h1-eth0' interface (IP: 10.0.0.1) and the 'lo' loopback interface (IP: 127.0.0.1). Both interfaces are up and running.

```
root@mininet-vm:/home/mininet# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 2a:0a:82:b0:8d:e1 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1366 bytes 170400 (170.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1366 bytes 170400 (170.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet#

root@mininet-vm:/home/mininet# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether c6:ed:47:60:d8:70 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1276 bytes 162328 (162.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1276 bytes 162328 (162.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet#
```

Рис. 4.1: Информацию о сетевых интерфейсах и IP-адресах хостов

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -с 6 (рис. 4.2).


```
host: h2"@"mininet-vm
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
loop txqueuelen 1000 (Local Loopback)
RX packets 1366 bytes 170400 (170.4 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1366 bytes 170400 (170.4 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=4.17 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.217 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.052 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.034 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.050 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.082 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5087ms
rtt min/avg/max/ndev = 0.034/0.767/4.172/1.523 ms
root@mininet-vm:/home/mininet#

MAGIC-COOKIE-1 9619d1d4c7047ef5960e725dac8319eb

host: h1"@"mininet-vm
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
loop txqueuelen 1000 (Local Loopback)
RX packets 1276 bytes 162328 (162.3 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1276 bytes 162328 (162.3 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.27 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.052 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.054 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.043 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5084ms
rtt min/avg/max/ndev = 0.043/0.425/2.270/0.825 ms
```

Рис. 4.2: Проверка соединения между хостами

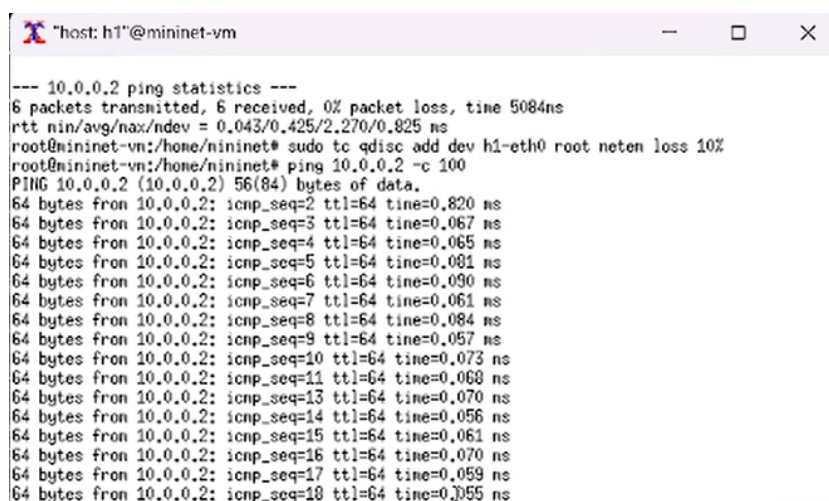
Пакеты могут быть потеряны в процессе передачи из-за таких факторов, как битовые ошибки и перегрузка сети. Скорость потери данных часто измеряется как процентная доля потерянных пакетов по отношению к количеству отправленных пакетов. На хосте h1 добавим 10% потерь пакетов к интерфейсу h1-eth0:

```
sudo tc qdisc add dev h1-eth0 root netem loss 10%
```

Здесь:

- sudo: выполнить команду с более высокими привилегиями;
- tc: вызвать управление трафиком Linux;
- qdisc: изменить дисциплину очередей сетевого планировщика;
- add: создать новое правило;
- dev h1-eth0: указать интерфейс, на котором будет применяться правило;
- netem: использовать эмулятор сети;
- loss 10%: 10% потерь пакетов.

Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду ping с параметром -c 100 с хоста h1. Параметр -c указывает общее количество пакетов для отправки. (рис. 4.3).



```

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5084ns
rtt min/avg/max/mdev = 0.043/0.425/2.270/0.825 ns
root@mininet-vn:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem loss 10%
root@mininet-vn:/home/mininet# ping 10.0.0.2 -c 100
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.820 ns
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.067 ns
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.065 ns
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.081 ns
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.090 ns
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.061 ns
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.084 ns
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.057 ns
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.073 ns
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.068 ns
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.070 ns
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.056 ns
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.061 ns
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.070 ns
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.059 ns
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.055 ns

```

Рис. 4.3: Добавление потери пакетов

Для эмуляции глобальной сети с потерей пакетов в обоих направлениях необходимо к соответствующему интерфейсу на хосте h2 также добавить 10% потерь пакетов. Проверим, что соединение между хостом h1 и хостом h2 имеет больший процент потерянных данных (10% от хоста h1 к хосту h2 и 10% от хоста h2 к хосту h1), повторив команду ping с параметром -c 100 на терминале хоста h1 (рис. 4.4).

```
root@mininet-vn: /home/mininet# sudo tc qdisc add dev h2-eth0 root netem loss 10%
root@mininet-vn: /home/mininet# ping 10.0.0.1 -c 100
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
 64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.922 ms
 64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.433 ms
 64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.239 ms
 64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.082 ms
 64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.072 ms
 64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=0.066 ms
 64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=0.065 ms
 64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=0.054 ms
 64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=0.057 ms
 64 bytes from 10.0.0.1: icmp_seq=13 ttl=64 time=0.066 ms
 64 bytes from 10.0.0.1: icmp_seq=14 ttl=64 time=0.071 ms
 64 bytes from 10.0.0.1: icmp_seq=16 ttl=64 time=0.068 ms
 64 bytes from 10.0.0.1: icmp_seq=17 ttl=64 time=0.060 ms
 64 bytes from 10.0.0.1: icmp_seq=18 ttl=64 time=0.069 ms
 64 bytes from 10.0.0.1: icmp_seq=19 ttl=64 time=0.066 ms
 64 bytes from 10.0.0.1: icmp_seq=21 ttl=64 time=0.097 ms
 64 bytes from 10.0.0.1: icmp_seq=22 ttl=64 time=0.059 ms
 64 bytes from 10.0.0.1: icmp_seq=23 ttl=64 time=0.049 ms
 64 bytes from 10.0.0.1: icmp_seq=24 ttl=64 time=0.138 ms
--- 10.0.0.1 ping statistics ---
 10 packets transmitted, 10 received, 0% packet loss, time 1004ms
rtt min/avg/max/ndev = 0.034/0.767/4.172/1.523 ms

root@mininet-vn: /home/mininet# sudo tc qdisc add dev h1-eth0 root netem loss 10%
root@mininet-vn: /home/mininet# ping 10.0.0.2 -c 100
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
 4 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.820 ns
 4 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.067 ns
 4 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.065 ns
 4 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.081 ns
 4 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.090 ns
 4 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.061 ns
 4 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.084 ns
 4 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.057 ns
 4 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.073 ns
 4 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.068 ns
 4 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.070 ns
 4 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.056 ns
 4 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.061 ns
 4 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.070 ns
 4 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.059 ns
 4 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.055 ns
--- 10.0.0.2 ping statistics ---
 18 packets transmitted, 6 received, 66% packet loss, time 5084ns
rtt min/avg/max/ndev = 0.043/0.425/2.270/0.825 ms
```

Рис. 4.4: Добавление потери пакетов

Добавим на интерфейсе узла h1 коэффициент потери пакетов 50% (такой высокий уровень потери пакетов маловероятен), и каждая последующая вероятность зависит на 50% от последней: Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду ping с параметром -c 50 с хоста h1 (рис. 4.5).

```
host: h1@mininet-vm
--- 10.0.0.2 ping statistics ---
100 packets transmitted, 81 received, 19% packet loss, time 101369ns
rtt min/avg/max/ndev = 0.043/0.076/0.820/0.085 ns
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.75 ns
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.054 ns
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.065 ns
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.047 ns
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.054 ns
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.057 ns

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5095ns
rtt min/avg/max/ndev = 0.047/0.671/3.751/1.377 ns
root@mininet-vm:/home/mininet# ^C
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem loss 50% 50%
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 50
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.79 ns
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.683 ns
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.161 ns
```

Рис. 4.5: Добавление значения корреляции для потери пакетов

Добавим на интерфейсе узла h1 0,01% повреждения пакетов. Проверим конфигурацию с помощью инструмента iPerf3 для проверки повторных передач (рис. 4.6).

```
host: h2@mininet-vm
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)

-----

^Croot@mininet-vm:/home/mininet#
root@mininet-vm:/home/mininet# iperf3 -s
warning: this system does not seem to support IPv6 - trying IPv4

Server listening on 5201

Accepted connection from 10.0.0.1, port 59628
[ 7] local 10.0.0.2 port 5201 connected to 10.0.0.1 port 59630
ID Interval Transfer Bitrate
[ 7] 0.00-1.00 sec 2.01 GBytes 17.2 Gbits/sec
[ 7] 1.00-2.00 sec 2.46 GBytes 21.2 Gbits/sec
[ 7] 2.00-3.00 sec 2.81 GBytes 24.0 Gbits/sec
[ 7] 3.00-4.00 sec 2.85 GBytes 24.6 Gbits/sec
[ 7] 4.00-5.00 sec 2.85 GBytes 24.5 Gbits/sec
[ 7] 5.00-6.00 sec 2.75 GBytes 23.6 Gbits/sec
[ 7] 6.00-7.00 sec 2.78 GBytes 23.9 Gbits/sec
[ 7] 7.00-8.00 sec 2.74 GBytes 23.6 Gbits/sec
[ 7] 8.00-9.00 sec 2.93 GBytes 25.2 Gbits/sec

-----

ast login: T
mininet@mininet-vm:~$
mininet-vm/unclear$
mininet@mininet-vm:/home/mininet#
mininet@mininet-vm:/home/mininet#
oot@mininet-vm:/home/mininet#
dac8319eb$
oot@mininet-vm:/home/mininet#
mininet@mininet-vm:/home/mininet#
--- 10.0.0.2 ping statistics ---
: command not found
50 packets transmitted, 27 received, 46% packet loss, time 50134ms
mininet@mininet-vm:/home/mininet#
rtt min/avg/max/ndev = 0.045/0.135/2.786/0.521 ms
** Creating root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
** Adding corroot@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem corrupt 0.01%
** Adding hoot@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
l h2
[ 7] local 10.0.0.1 port 59630 connected to 10.0.0.2 port 5201
** Adding swID Interval Transfer Bitrate Retr Cwnd
l 7] 0.00-1.00 sec 2.02 GBytes 17.3 Gbits/sec 12 1.51 MBytes
** Adding li7] 1.00-2.00 sec 2.47 GBytes 21.2 Gbits/sec 3 1.45 MBytes
h1, s1) (h2,7] 2.00-3.00 sec 2.80 GBytes 24.0 Gbits/sec 8 2.17 MBytes
7] 3.00-4.00 sec 2.86 GBytes 24.5 Gbits/sec 7 2.33 MBytes
** Configur7] 4.00-5.00 sec 2.87 GBytes 24.7 Gbits/sec 9 649 KBytes
l h27] 5.00-6.00 sec 2.75 GBytes 23.6 Gbits/sec 8 604 KBytes
** Running t7] 6.00-7.00 sec 2.78 GBytes 23.9 Gbits/sec 3 716 KBytes
** Starting7] 7.00-8.00 sec 2.74 GBytes 23.5 Gbits/sec 5 675 KBytes
07] 8.00-9.00 sec 2.93 GBytes 25.2 Gbits/sec 6 646 KBytes
```

Рис. 4.6: Добавление повреждения пакетов

Добавим на интерфейсе узла h1 следующее правило: 25% пакетов (со значением корреляции 50%) будут отправлены немедленно, а остальные 75% будут задержаны на 10 мс. Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду `ping` с параметром `-c 20` с хоста h1. Убедимся, что часть пакетов не будут иметь задержки (один из четырех, или 25%), а следующие несколько пакетов будут иметь задержку около 10 миллисекунд (три из четырех, или 75%) (рис. 4.7).


```

host: h1@mininet-vm
[ 7] 6,00-7,00 sec 2,78 GBytes 25,9 Gbits/sec 3 716 KBytes
[ 7] 7,00-8,00 sec 2,74 GBytes 25,5 Gbits/sec 5 675 KBytes
[ 7] 8,00-9,00 sec 2,93 GBytes 25,2 Gbits/sec 6 646 KBytes
[ 7] 9,00-10,00 sec 2,52 GBytes 21,6 Gbits/sec 5 662 KBytes
-----
[ ID] Interval      Transfer      Bitrate      Retr
[ 7] 0,00-10,00 sec 26,7 GBytes 23,0 Gbits/sec 66
[ 7] 0,00-10,00 sec 26,7 GBytes 22,9 Gbits/sec
sender
receiver

iperf Done.
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 10ms reorder 25% 50%
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=11,7 ns
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0,399 ns
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=11,2 ns
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=10,4 ns
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=10,5 ns
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=10,2 ns
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=10,3 ns
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=10,7 ns
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=10,8 ns
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=10,4 ns
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=10,2 ns

```

Рис. 4.7: Добавление переупорядочивания пакетов

Для интерфейса узла h1 зададим правило с дублированием 50% пакетов (т.е. 50% пакетов должны быть получены дважды): Проверим, что на соединении от хоста h1 к хосту h2 имеются дублированные пакеты, используя команду ping с параметром -c 20 с хоста h1. Дубликаты пакетов помечаются как DUP! (рис. 4.8).

```

host: h1@mininet-vm
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=10,3 ns

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19004ms
rtt min/avg/max/mdev = 0,083/9,558/11,710/3,133 ns
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem duplicate 50%
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2,81 ns
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3,82 ns (DUP!)
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0,640 ns
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1,14 ns (DUP!)
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0,183 ns
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0,086 ns
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0,086 ns (DUP!)
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0,132 ns
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0,132 ns (DUP!)
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0,059 ns
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0,125 ns
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0,126 ns (DUP!)
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0,057 ns
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0,058 ns (DUP!)
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0,061 ns
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0,061 ns (DUP!)

```

Рис. 4.8: Добавление дублирования пакетов

В виртуальной среде mininet в своём рабочем каталоге с проектами создадим каталог simple-dgor и перейдем в него. Создадим скрипт для эксперимента lab_netem_ii.py (рис. 4.9).

```

Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )
    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s1 = net.addSwitch( 's1' )

    info( '*** Creating links\n' )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )

    info( '*** Starting network\n' )
    net.start()

    info( '*** Set delay\n' )
    h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem loss 10%' )
    h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 10%' )

    time.sleep(10) # Wait 10 seconds

    info( '*** Ping\n' )
    h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" |

```

Рис. 4.9: Создание скрипта для эксперимента lab_netem_ii.py

Скорректируем скрипт так, чтобы на экран или в отдельный файл выводилась информация о потерях пакетов (рис. 4.10).

```

info( '*** Ping\n' )
h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" |
awk \'/print $5, $7\)/' | sed -e \'/s/time=//g\' -e

```

Рис. 4.10: Редактирование скрипта

Создадим Makefile для управления процессом проведения эксперимента (рис. 4.11).

```
mininet@mininet-vm: ~/work/lab_netem_ii/simple-drop
GNU nano 4.8 Makefile
all: ping.dat
ping.dat:
    sudo python lab_netem_ii.py
    sudo chown mininet:mininet ping.dat
clean:
    -rm -f *.dat
```

Рис. 4.11: Makefile

Выполним эксперимент (рис. 4.12).

```
make: *** Makefile:3: ping.dat Error 1
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ ^C
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ sudo rm -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noses
Killall -9 controller ofprotocol ofdatapath ping nos_core lt-nos_core ovs-openflowd ovs-controller ovs-testcontroller udptsttest mnetec ivs ryu-manager 2> /dev/null
killall -9 -f "sudo mnetec"
*** Removing junk from /tmp
rm -f /tmp/vnodes /tmp/vlogs /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]*' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --if-exists del-br s1
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([f-...[a-zm]]+)-eth[0-9]+'
(ip link del s1-eth1 ip link del s1-eth2 ) 2> /dev/null
ip link show
ip link show
*** Killing stale mininet node processes
killall -9 -f mininet:
*** Shutting down stale tunnels
killall -9 -f Tunnel=Ethernet
killall -9 -f ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ make
sudo python lab_netem_ii.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
C0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Setting packet loss
*** h1 : ('tc qdisc add dev h1-eth0 root netem loss 10%')
*** h2 : ('tc qdisc add dev h2-eth0 root netem loss 10%')
*** Running ping test
*** h1 : ('ping -c 10 10.0.0.2 | grep "time" | awk '{print $7, $8}' | sed -e 's/time=/g/' > ping.dat')
```

Рис. 4.12: Проведение эксперимента

5 Выводы

В результате выполнения данной лабораторной работы я получила навыки проведения интерактивных экспериментов в среде Mininet по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных.

Список литературы